

PRUEBA TÉCNICA – CRUD BACKEND (SENIOR) CON JAVA + SPRING BOOT

Tiempo sugerido: 1 hora

Lenguaje: Java 17

Framework: Spring Boot

Alcance: Solo Backend

Objetivo

Construir un microservicio REST en Java usando Spring Boot que implemente un CRUD completo con calidad de producción (arquitectura, validaciones, errores, pruebas, documentación y despliegue).

Dominio

- Entidad principal: Cliente (tabla: clientes)
- id (UUID, PK)
- nombre (String, obligatorio, min 3)
- email (String, obligatorio, formato válido, único)
- telefono (String, opcional, si viene 7 a 15 dígitos)
- activo (boolean, default true)
- deletedAt (LocalDateTime, null si no está borrado) — borrado lógico
- createdAt (LocalDateTime, automático)
- updatedAt (LocalDateTime, automático)

Endpoints Requeridos

- POST /api/clientes → crear (201)
- GET /api/clientes → listar (200) con filtros + paginación + orden
- GET /api/clientes/{id} → consultar (200 / 404)
- PUT /api/clientes/{id} → actualizar total (200 / 404 / 409 / 412)
- PATCH /api/clientes/{id} → actualización parcial (200 / 404 / 409 / 412)
- DELETE /api/clientes/{id} → borrado lógico (204 / 404)
- PATCH /api/clientes/{id}/activar?value=true|false → activar/desactivar (200 / 404)

Reglas de Negocio y Validaciones

- Email único (case-insensitive)
- No permitir operaciones sobre registros borrados (deletedAt != null)
- Listar por defecto solo no borrados; permitir incluir borrados con query param includeDeleted=true
- Filtro q: busca en nombre o email (contains, case-insensitive)
- Filtro activo: activo=true|false (opcional)
- Concurrencia: si hay conflicto de versión (optimistic lock) devolver 412 Precondition Failed o 409, pero documentar la decisión
- Idempotencia DELETE: si no existe → 404 (obligatorio)

DTOs – Requeridos

Definir DTOs mínimos:

- ClienteCreateRequest { nombre, email, teléfono }
- ClienteUpdateRequest { nombre, email, teléfono, activo, versión } (versión requerida)
- ClientePatchRequest (campos opcionales) + versión requerida
- ClienteResponse { id, nombre, email, teléfono, activo, createdAt, updatedAt, versión }
- PageResponse<T> (opcional) si no usas Page directamente

Manejo de Errores

Todas las respuestas de error deben tener este formato:

```
{  
  "timestamp": "2026-01-30T12:00:00Z",  
  "status": 409,  
  "error": "Conflict",  
  "message": "El email ya está registrado",  
  "path": "/api/clientes"  
}
```

Códigos mínimos esperados:

- 400 → validaciones
- 404 → no encontrado
- 409 → duplicidad (email)
- 412 (o 409) → conflicto de versión (optimistic locking)
- 500 → error inesperado (sin filtrar stacktrace al cliente)

Documentación y Ejecutable

- README con: requisitos, cómo correr local, cómo correr con Docker, variables de entorno
- Incluir ejemplos curl o colección Postman

Despliegue (Obligatorio)

- Dockerfile para la API
- docker-compose.yml levantando API + PostgreSQL
- Variables por entorno: DB_URL, DB_USER, DB_PASS, SERVER_PORT
- Comando único esperado: docker compose up --build